사용자 정의 함수와 제어 구문

이청용 교수
leecy@hansung.ac.kr

강의개요

- **사용자 정의 함수**
  - Python에서 함수
  - 함수 작성 방법
  - 익명(anonymous) 함수
  - 위치/키워드 인자
  - 매개변수 기본값 지정하기
  - 위치/키워드 인자 취합하기
- **조건문**
  - 제어구문
  - 비교 연산자와 논리 연산자
  - if – else 조건문
  - 예제: is.even( )
  - 다중 선택 조건문
- **반복문**
  - for 반복문
  - 예제: sum.num( )
  - 반복문의 제어
  - while 반복문

사용자 정의 함수

Python에서 함수

- Python에서 모든 연산은 함수 호출에 의해 실행됨
  - 함수는 하나 이상의 입력값(매개변수, parameter)과 함께 실행되며, 실행 결과로 하나의 결과값(객체, object)을 반환함
  - 함수에서 실행되는 연산은 입력값에만 영향을 받기 때문에 숨겨진 입력이나 출력에 의한 부작용을 줄일 수 있음 → **순수(pure) 함수**

```python
# 함수 정의
def foo(x):
    return x + 10
```

Python

```python
# 인자 없이 실행 → 오류 발생
foo()
```

Python

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 foo()

TypeError: foo() missing 1 required positional argument: 'x'
```

함수 작성 방법

- 일반적으로 Python 함수는 다음과 같이 정의됨
  - arglist: 함수에 사용될 인자의 이름과 기본값을 정의하는 부분으로, 각 인자는 쉼표로 구분됨
  - body: 함수의 기능을 정의하는 부분으로, 함수 실행을 통해 반환되는 값을 결정함

```python
def 함수명(arglist):
    body
```

- 일반적으로 함수에는 이름이 부여되며, 해당 이름을 통해 다른 구문에서 재사용이 가능함
  - 필요에 따라 이름 없이 함수를 정의할 수도 있음 → 익명(anonymous) 함수
  - 함수 내에서 실행해야 하는 연산이 복잡한 경우에는 여러 개의 구문을 결합할 수 있음

함수 작성 방법

- 다음은 입력된 인자의 제곱을 계산하는 간단한 함수 정의 예시임
  - 사용자 정의 함수도 일반적인 다른 함수와 동일한 방식으로 사용 가능함
  - 연산자 *는 벡터 객체의 요소별로 적용되므로, square( ) 함수도 동일한 방식으로 연산 수행 가능함

```python
# 제곱 계산 함수 정의
def square(x):
    return x * x
```

Python

```python
# 함수 호출
print(square(10))
```

Python

```
100
```

함수 작성 방법

- 앞서 정의한 square( ) 함수는 다른 함수 내부에서 동일하게 사용 가능함
  - 아래 예시는 square( ) 함수를 내부에서 재사용하여 1부터 x까지의 제곱 합을 계산함

```python
# 제곱의 합을 계산하는 함수 정의
def sumsq(x):
    temp = []
    for i in range(1, x + 1):
        temp_sq = square(i)   # square() 함수 재사용
        temp.append(temp_sq)
    return sum(temp)
```

Python

```python
# 함수 호출
print(sumsq(10))
```

Python

```
385
```

익명(anonymous) 함수

- 일반적으로 함수에는 이름을 부여하지만, lambda 키워드를 사용하면 이름 없이 익명으로 정의할 수 있음
  - lambda 매개변수: 표현식 형태로 작성하며, 표현식의 결과가 반환값이 됨

```python
# 일반 함수 정의
def add(x, y):
    return x + y
```

Python

```python
# 함수 호출
print(add(10, 20))
```

Python

```
30
```

```python
# 익명 함수 정의 및 호출
print((lambda x, y: x + y)(10, 20))
```

Python

```
30
```

익명(anonymous) 함수

- 익명 함수는 다양한 용도로 사용될 수 있으며, 특히 다른 함수에서 인자로 사용되는 경우가 많음
  - 주로 정렬(sorting)이나 데이터 변환(map, filter 등)에서 자주 사용됨

```python
# 튜플(이름:나이) 리스트 정의
students = [("Kim", 22), ("Lee", 19), ("Park", 25)]

# 나이를 기준으로 정렬
students.sort(key=lambda x: x[1])
print(students)
```

Python

```
[('Lee', 19), ('Kim', 22), ('Park', 25)]
```

위치 인자(positional arguments)

- Python은 함수의 인자(arguments)를 유연하고 독특하게 처리함
  - 인자의 가장 익숙한 타입은 값을 순서대로 대응하는 매개변수(parameter)에 복사하는 위치 인자임

```python
# 함수 정의
def menu(x1, x2, x3):
    return {"a": x1, "b": x2, "c": x3}
```

Python

```python
# 함수 출력
print(menu(1, 2, 3))
```

Python

```
{'a': 1, 'b': 2, 'c': 3}
```

- 위치 인자의 단점은 각 위치의 의미를 반드시 알고 있어야 한다는 점임
  - 인자의 위치가 바뀌면 함수의 결과도 달라짐

```python
# 함수 출력 → 인자 위치 변경
print(menu(2, 1, 3))
```

Python

```
{'a': 2, 'b': 1, 'c': 3}
```

키워드 인자(keyword arguments)

- 위치 인자의 혼동을 피하기 위해 매개변수에 상응하는 이름을 인자에 지정할 수 있음
  - 키워드 인자는 함수 정의와 다른 순서로 지정할 수 있음
  - 위치 인자와 키워드 인자를 함께 사용할 경우, 위치 인자가 먼저 와야 함

```python
# 함수 정의
def menu(x1, x2, x3):
    return {"a": x1, "b": x2, "c": x3}
```

Python

```python
# 키워드 인자 사용 (순서 무관)
print(menu(x3="x3", x2="x2", x1="x1"))
```

Python

```
{'a': 'x1', 'b': 'x2', 'c': 'x3'}
```

```python
# 위치 인자 + 키워드 인자 혼합
print(menu("x1", x3="x3", x2="x2"))
```

Python

```
{'a': 'x1', 'b': 'x2', 'c': 'x3'}
```

매개변수 기본값 지정하기

- 매개변수에 기본값을 지정할 수 있으며, 호출자가 해당 인자를 제공하지 않으면 기본값을 사용함

```python
# 함수 정의 → 기본값 지정
def menu(x1, x2, x3=3):
    return {"a": x1, "b": x2, "c": x3}
```

Python

```python
# x3를 생략 → 기본값 3 사용
print(menu(1, 2))
```

Python

```
{'a': 1, 'b': 2, 'c': 3}
```

- 인자를 입력하면 기본값 대신 입력한 인자가 사용됨

```python
# x3를 입력 → 입력값 사용
print(menu(1, 2, 4))
```

Python

```
{'a': 1, 'b': 2, 'c': 4}
```

위치 인자 취합하기

- 함수의 매개변수에 애스터리스크(*)를 사용하면 위치 인자들을 튜플(tuple)로 묶어줌
  - 관례적으로 매개변수 이름은 args를 사용함

```python
# 함수 정의
def sumsq(*args):
    result = 0
    for i in args:
        result += i
    return result
```

Python

```python
# 여러 개의 인자를 전달 가능
print(sumsq(1, 2, 3, 4, 5))
```

Python

```
15
```

키워드 인자 취합하기

- 키워드 인자를 딕셔너리로 묶기 위해 두 개의 애스터리스크(**)를 사용함
  - 인자의 이름은 키(key), 값은 해당 키에 대응하는 딕셔너리 값(value)을 나타냄

```python
# 함수 정의
def print_kwargs(**kwargs):
    print("Keyword arguments:", kwargs)
```

Python

```python
# 키워드 인자를 딕셔너리 형태로 묶어서 전달
print_kwargs(a=1, b=2, c=3)
```

Python

```
Keyword arguments: {'a': 1, 'b': 2, 'c': 3}
```

조건문

제어 구문

- Python을 포함한 대부분의 컴퓨터 프로그램 코드는 일반적으로 위에서 아래로, 왼쪽에서 오른쪽으로 순차적으로 실행됨
  - 이러한 일방적인(unidirectional) 흐름은 복잡한 과업 수행 시 비효율적일 수 있음
- 프로그래밍 언어에서 제어 구문은 실행 순서를 변경하거나 조건에 따라 실행해야 할 명령문을 제어할 때 사용함
  - 특정 조건을 만족하는 경우에만 실행(조건문)하거나, 일정 부분을 반복하여 실행(반복문)하도록 제어할 수 있음
  - 제어 구문을 통해 순차(sequence), 선택(selection), 반복(iteration) 의 논리 구조를 조합하여 프로그램을 구성함

제어 구문

- Python의 제어 구문은 다른 프로그래밍 언어와 동일하게 사용됨
  - 조건문(선택 제어): if – else
  - 반복문(반복 제어): for, while, break, continue
- 제어 구문은 대부분 함수 내부에서 사용됨
  - 콘솔에서 단순 명령을 하나씩 입력해 결과를 확인하는 경우에는 제어 구문의 필요성이 크지 않음
  - 복잡한 흐름 제어를 위해 프로그램 내에서 조건문과 반복문을 활용함

```python
# 조건문 기본 구조
x = 10

if x > 0:
    print("양수임")
elif x == 0:
    print("0임")
else:
    print("음수임")
```

Python

양수임

비교 연산자와 논리 연산자

- Python에서 사용할 수 있는 비교 연산자와 논리 연산자는 다음과 같음

연산자	의미
〈	작다
〈=	작거나 같다
〉	크다
〉=	크거나 같다
==	같다
!=	같지 않다
or	논리 연산자(또는)
and	논리 연산자(그리고)
not	논리 연산자(부정)
in	포함 여부 확인

if – else 조건문

- 선택 제어문 if – else는 주어진 조건의 참(Ture)/거짓(False) 여부에 따라 두 개의 구문 중 하나만 실행됨

```python
if condition:
    expr1
else:
    expr2
```

- 주어진 condition이 참이면 expr1을 실행하고, 거짓이면 expr2를 실행함

```python
x = 5

if x > 0:
    print("Big")
else:
    print("Small")
```

```
Big
```

if – else 조건문

- 조건이 거짓일 때 실행할 else 구문이 필요하지 않은 경우, if만 단독으로 사용 가능함 → 단순 조건문

```python
if condition:
    expression
```

- 이는 사실상 다음 구문과 동일한 의미임

```python
if condition:
    expression
else:
    None
```

예제: is_even( )

- 아래의 is_even( ) 함수는 사용자가 입력한 숫자가 짝수인지 홀수인지를 판단함
  - 입력값이 짝수인 경우 True 반환함
  - 입력값이 홀수인 경우 False 반환함

```python
# 함수 정의
def is_even(x):
    if x % 2 == 0:
        return True
    else:
        return False
```

Python

```python
# 함수 호출
print(is_even(5))
```

Python

```
False
```

예제: is_even( )

- 단순히 True, False를 반환하는 것보다 입력값이 홀수인지 짝수인지를 직접 출력하면 더 유용할 수 있음

```python
# 함수 정의
def is_even(x):
    if x % 2 == 0:
        print(x, "is an even number")
    else:
        print(x, "is an odd number")
```

Python

```python
# 함수 호출
is_even(4)
```

Python

```
4 is an even number
```

예제: is_even( )

- 입력값이 정수가 아닐 경우 의도치 않은 결과가 발생할 수 있음

```python
# 입력 값이 정수 → 정상 출력
is_even(4)
```

Python

```
4 is an even number
```

```python
# 입력 값이 정수가 아닌 경우 → 오류
is_even(3.5)
```

Python

```
3.5 is an odd number
```

다중 선택 조건문

- if – else 구문을 중첩하여 사용하면 조건식을 순차적으로 비교하는 다중 선택 구조를 만들 수 있음

```python
if condition1:
    if sub_condition:
        expr1
    else:
        expr2
else:
    expr3
```

다중 선택 조건문

- 조건문의 중첩을 통해 is_even( ) 함수를 다음과 같이 수정하면 앞서 살펴본 문제점을 제거할 수 있음

```python
# 함수 정의
def is_even(x):
    if x % 1 == 0 and x > 0:
        if x % 2 == 0:
            print(x, "is an even number")
        else:
            print(x, "is an odd number")
    else:
        print("NA")
```

Python

```python
# 함수 호출
is_even(3.5)
```

Python

```
NA
```

반복문

for 반복문

- 반복문은 프로그램 내에서 동일한 명령을 일정 횟수만큼 또는 주어진 조건을 만족하는 동안 반복하여 수행하도록 하는 구조임
  - Python에서는 for, while 문을 사용하여 반복문을 구성
- for 반복문의 기본 구조는 다음과 같음
  - 반복문 내의 expression은 list, tuple, string의 길이만큼 반복하여 실행함
  - 각 시행마다 variable에는 list, tuple, string의 원소 값이 하나씩 순서대로 입력함

```python
for variable in list(or tuple, string):
    expression
```

for 반복문

- 반복문을 실행했을 때 나타나는 결과는 아래와 같음

```python
# 리스트 생성
nums = [1, 2, 3, 4, 5]

# 리스트의 각 원소에 대해 반복
for i in nums:
    print("The No. is", i)
```

```
The No. is 1
The No. is 2
The No. is 3
The No. is 4
The No. is 5
```

- 만약 nums에 [(1, 2), (3, 4)]와 같이 튜플을 넣어주면, 반복문에서 각 요소가 튜플 단위로 처리함

```python
# 튜플 리스트 생성
nums = [(1, 2), (3, 4)]

# 리스트의 각 튜플에 대해 반복
for i in nums:
    print("The tuple is", i)
```

```
The tuple is (1, 2)
The tuple is (3, 4)
```

for 반복문

- 다음과 같이 직접 실행 횟수를 정해줄 수도 있음

```python
# 리스트의 각 원소에 대해 반복
for i in list(range(1, 8))[::-1]:
    print("The No. is", i)
```

Python

```
The No. is 7
The No. is 6
The No. is 5
The No. is 4
The No. is 3
The No. is 2
The No. is 1
```

- 반복문에서는 숫자 리스트를 자동으로 만들어주는 range( ) 함수와 함께 자주 사용함

```python
# 1부터 7까지의 정수 리스트 생성
list(range(1, 8))
```

Python

```
[1, 2, 3, 4, 5, 6, 7]
```

예제: sum_num( )

- 다음은 주어진 벡터 x에 포함된 숫자의 합을 계산하는 함수 sum_num( ) 정의 예시임
  - Python 내장 함수 sum( )으로도 구현할 수 있지만, 여기서는 반복문을 사용하여 직접 구현함

```python
# 함수 정의
def sum_num(x):
    result = 0
    for i in range(x + 1):
        result = result + i
    print("Total:", result)
```

Python

```python
# 함수 호출
sum_num(10)
```

Python

```
Total: 55
```

반복문의 제어

- 반복문 내에서 특정 조건을 만족하는 경우, 반복문 내 일부를 실행하지 않거나 반복 자체를 종료하도록 설정할 수 있음
  - 반복문 내에서 continue 또는 break가 나오면 반복문의 실행을 건너뛰거나 반복 자체를 완전히 종료함
- 반복문 내에서 continue 사용 예시는 다음과 같음
  - 특정 조건을 만족하면 현재 반복을 건너뛰고 다음 반복을 수행함

```python
# 1부터 5까지 반복
for i in range(1, 6):
    # i가 3이면 출력하지 않고 다음 반복으로 넘어감
    if i == 3:
        continue
    # i 출력
    print(i)
```

Python

```
1
2
4
5
```

반복문의 제어

- 반복문 내에서 break는 continue와 다르게 반복 전체를 종료함

```python
# 1부터 5까지 반복
for i in range(1, 6):
    # i가 3이면 반복문 종료
    if i == 3:
        break
    # i 출력
    print(i)
```

Python

```
1
2
```

반복문의 제어

- 다음 예시는 continue와 break 의 차이를 명확히 보여줌
  - continue는 특정 조건에서 해당 반복만 건너뛰고 다음 반복을 실행함
  - break는 특정 조건에서 반복문 전체를 종료함

```python
# 1부터 10까지 반복
for i in range(1, 11):
    # i가 3이면 출력하지 않고 다음 반복으로 넘어감
    if i == 3:
        continue
    # i가 7이면 반복문 종료
    elif i == 7:
        break
    # i 출력
    print(i)
```

Python

```
1
2
4
5
6
```

반복문의 제어

- zip( ) 함수는 여러 리스트의 원소를 같은 인덱스끼리 묶어서 순회할 수 있음
  - 가장 짧은 리스트의 길이에 맞추어 반복이 종료됨

```python
# 이름과 점수 리스트 생성
names = ["Tom", "Jane", "Mike"]
scores = [90, 85, 95]

# 이름과 점수를 함께 출력
for name, score in zip(names, scores):
    print(name, "scored", score)
```

```
Tom scored 90
Jane scored 85
Mike scored 95
```

반복문의 제어

- enumerate( ) 함수는 순서가 있는 자료형(리스트, 튜플, 문자열)을 입력받아 인덱스와 값을 함께 반환함
  - 보통 for 문과 함께 사용되며, 반복되는 항목의 위치(인덱스)가 필요할 때 유용

```python
# 리스트 생성
fruits = ["apple", "banana", "cherry"]

# 인덱스와 원소를 함께 출력
for i, fruit in enumerate(fruits):
    print(i, fruit)
```

Python

```
0 apple
1 banana
2 cherry
```

while 반복문

- 지금까지 살펴본 for 반복문은 정해진 횟수(range의 길이)만큼 반복적으로 수행함
  - 만약 횟수를 지정하는 대신, 주어진 조건이 참일 동안 반복하고 싶을 때는 while 반복문을 사용함
- while 반복문은 조건에 따라 반복 수행이 가능함
  - 아래 반복문은 주어진 condition이 참(True)인 동안 expression이 반복 실행됨

```python
while condition:
    expression
```

while 반복문

- 아래 예시는 while 반복문을 사용하여 1부터 5까지 출력한 결과임

```python
# 초기값 설정
count = 1

# 1부터 5까지 반복
while count <= 5:
    # 현재 count 값 출력
    print(count)
    # count 값을 1 증가
    count += 1
```

Python

```
1
2
3
4
5
```

수고하셨습니다.